LAPORAN
WORKSHOP SISTEM INFORMASI
WEB SERVER
(TIF330805)
SEMESTER III



Acara 22

Nama:

Nila Agustin Prima Aurelia

NIM:

E41250666

Golongan:

A

PROGRAM STUDI DILUAR KAMPUS UTAMA (PSDKU)

TEKNIK INFORMATIKA KAB. NGANJUK

JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI JEMBER

TAHUN 2026

BAB 1 PENDAHULUAN

1.1 Latar Belakang

Pada Acara 21, RESTful API si-akademik yang dibangun dengan PHP Native (pola MVC, Router, dan Front Controller) telah didokumentasikan dalam berkas Markdown API_DOCUMENTATION.md serta fitur dokumentasi Postman. Dokumentasi Markdown mudah ditulis, tetapi tidak memiliki struktur baku, tidak dapat divalidasi secara otomatis, dan tidak dapat dipakai untuk menghasilkan halaman dokumentasi interaktif maupun kode klien.

Untuk mengatasi keterbatasan tersebut, industri menggunakan OpenAPI Specification, yaitu format standar untuk mendeskripsikan REST API dalam dokumen YAML atau JSON yang dapat dibaca manusia maupun mesin. Pada Acara 22 ini, dokumentasi Markdown Acara 21 dikonversi menjadi berkas openapi.yaml (OpenAPI 3.0), lalu divisualisasikan dengan Swagger Editor. Path pada dokumen mengikuti resource pada peta Router (/login, /register, /mahasiswa), bukan lagi nama berkas .php.

1.2 Tujuan Praktikum

1. Memahami standar OpenAPI Specification dan keunggulannya dibandingkan dokumentasi Markdown manual.
2. Menyusun dokumentasi endpoint si-akademik dalam berkas docs/openapi.yaml berdasarkan Router dan Controller yang telah dibangun.
3. Menambahkan komponen securitySchemes bertipe http/bearer untuk endpoint yang terproteksi token JWT.
4. Memvisualisasikan dokumentasi menggunakan Swagger Editor dan menyimpan hasilnya sebagai lampiran proyek.

BAB 2 DASAR TEORI

2.1 Standar OpenAPI Specification

OpenAPI Specification (dahulu dikenal sebagai Swagger Specification) merupakan format standar industri untuk mendeskripsikan REST API dalam dokumen YAML atau JSON yang dapat dibaca baik oleh manusia maupun mesin. Dibandingkan dokumentasi Markdown manual yang dibuat pada Acara 21, OpenAPI menawarkan keunggulan berupa struktur yang baku, dapat divalidasi secara otomatis, serta dapat digunakan untuk menghasilkan dokumentasi interaktif maupun kode klien secara otomatis.

Sejak tahun 2015, spesifikasi Swagger 2.0 didonasikan kepada OpenAPI Initiative di bawah Linux Foundation. Versi terbaru (OpenAPI 3.x) membawa berbagai penyempurnaan dibandingkan Swagger 2.0.

2.2 Struktur Dasar Dokumen OpenAPI

Dokumen OpenAPI 3.0 yang digunakan pada praktikum ini memiliki bagian utama berikut.

Bagian	Fungsi
openapi	Versi spesifikasi yang dipakai (3.0.0)
info	Judul, deskripsi, dan versi API
servers	Alamat dasar (base URL) API
paths	Daftar endpoint beserta method HTTP, parameter, request body, dan response
components	Komponen yang dapat dipakai ulang, seperti schemas, responses, parameters, dan securitySchemes
security	Penerapan skema keamanan pada endpoint tertentu

2.3 Swagger Editor dan Swagger UI sebagai Alat Visualisasi

Swagger Editor adalah alat berbasis web yang memungkinkan penulisan dokumen OpenAPI dengan pratinjau (preview) dokumentasi secara langsung. Swagger UI menghasilkan halaman dokumentasi interaktif yang memungkinkan pengguna mencoba langsung (try-it-out) setiap endpoint dari tampilan dokumentasi tanpa berpindah ke Postman.

2.4 Autentikasi Bearer Token (JWT)

Endpoint yang terproteksi pada si-akademik memerlukan token JWT (HS256) yang diperoleh dari POST /login dan berlaku selama 5 menit. Token dikirim pada header Authorization: Bearer <token>. Dalam OpenAPI, mekanisme ini dideklarasikan melalui securitySchemes bertipe http dengan scheme: bearer.

BAB 3 HASIL PRAKTIKUM

3.1 Prosedur Kerja

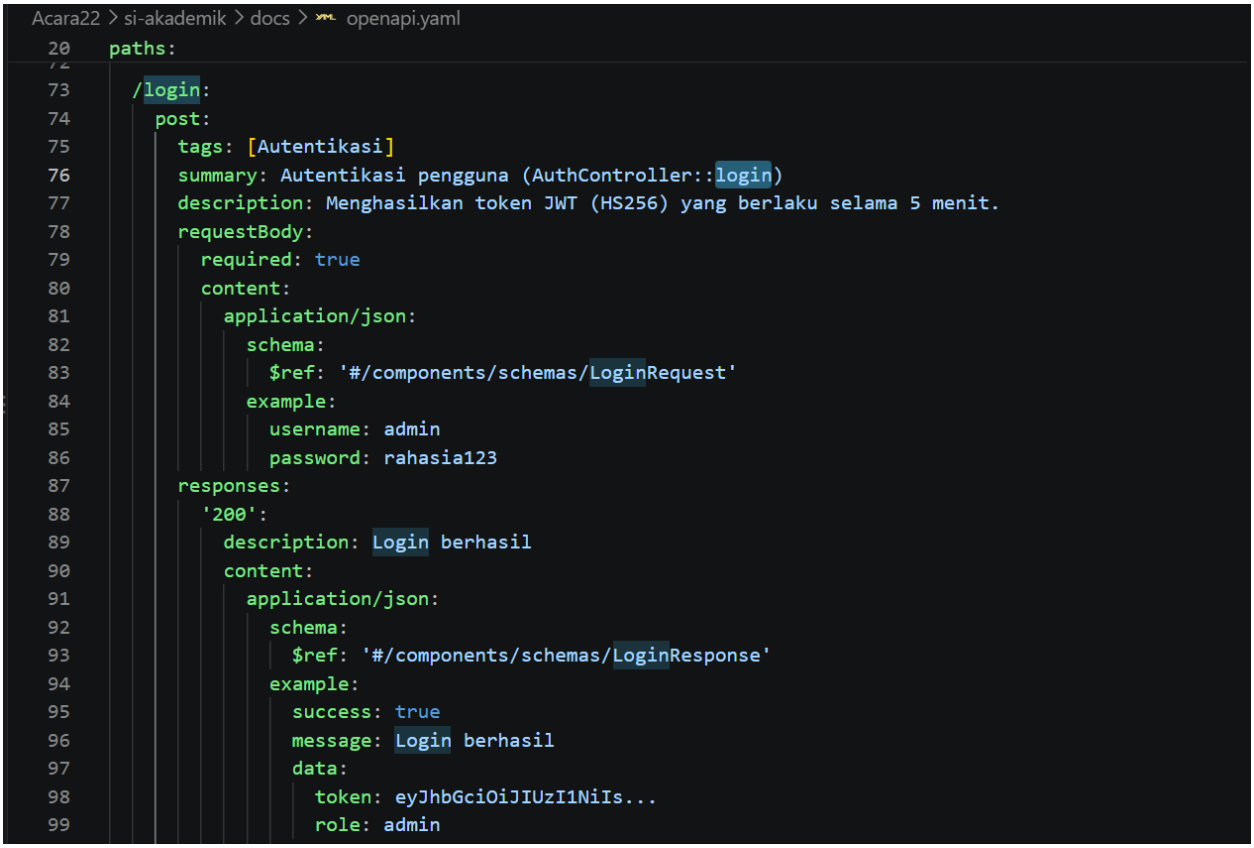
Dokumentasi Markdown Acara 21 dikonversi menjadi OpenAPI 3.0 berdasarkan Router dan Controller pada project si-akademik, yaitu AuthController (register, login) dan MahasiswaController (index, store, update, destroy). Berkas dibuat pada docs/openapi.yaml.

Langkah 1 - Membuat File openapi.yaml

Pada folder docs dibuat berkas openapi.yaml yang berisi kerangka awal: versi OpenAPI, informasi API, dan alamat server. Base URL disesuaikan dengan lokasi project pada server lokal.

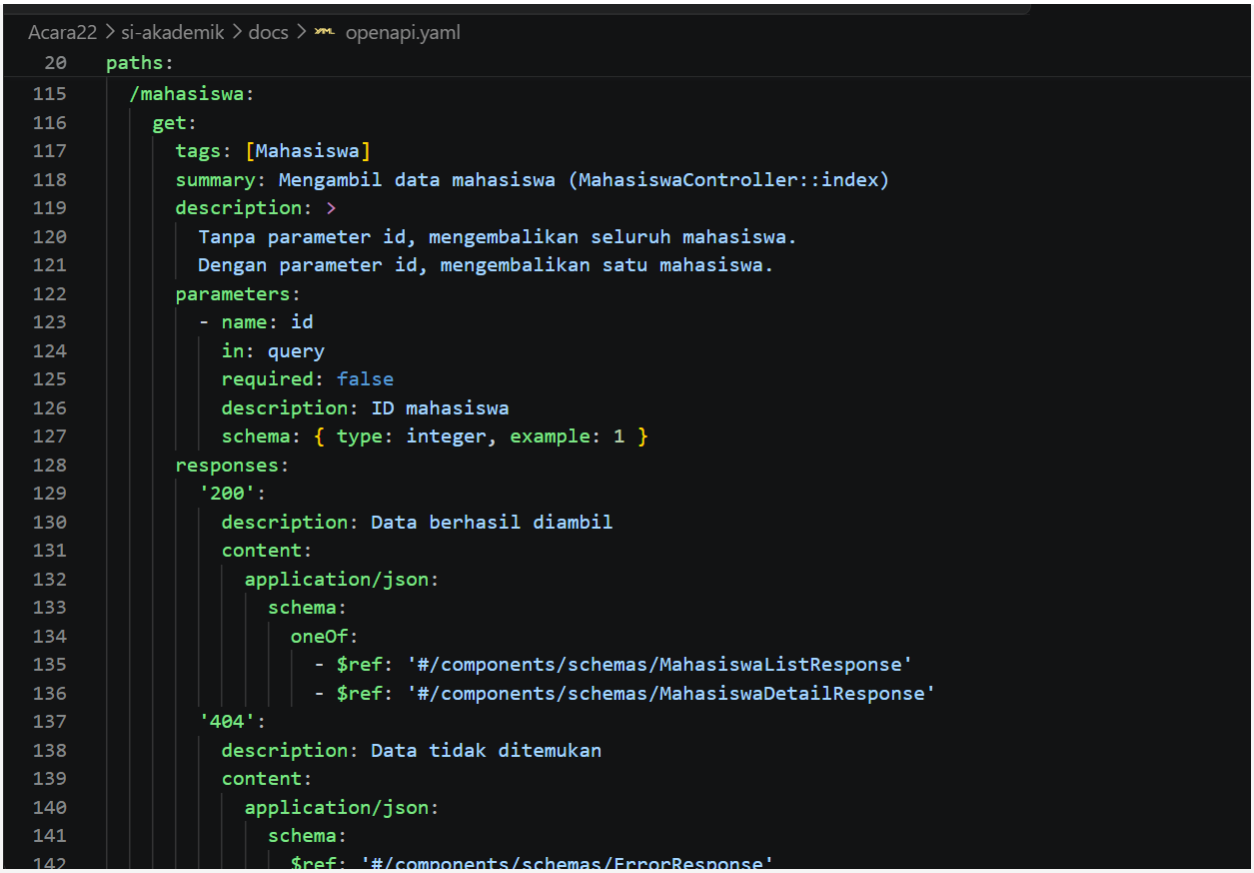
Langkah 2 - Mendefinisikan Path /login

Path /login hanya memiliki method post karena Router menolak method lain dengan kode 405. Request body berisi username dan password. Response yang didokumentasikan adalah 200 (berisi token dan role), 401 (kredensial salah), dan 405.



Langkah 3 - Mendefinisikan Path /mahasiswa

Path /mahasiswa mengikuti pemetaan Router: GET ke index, POST ke store, PUT ke update, dan DELETE ke destroy. ID dikirim melalui query string (?id=1). Method GET bersifat publik, sedangkan POST, PUT, dan DELETE diberi security: bearerAuth karena memanggil Auth::requireAuth().

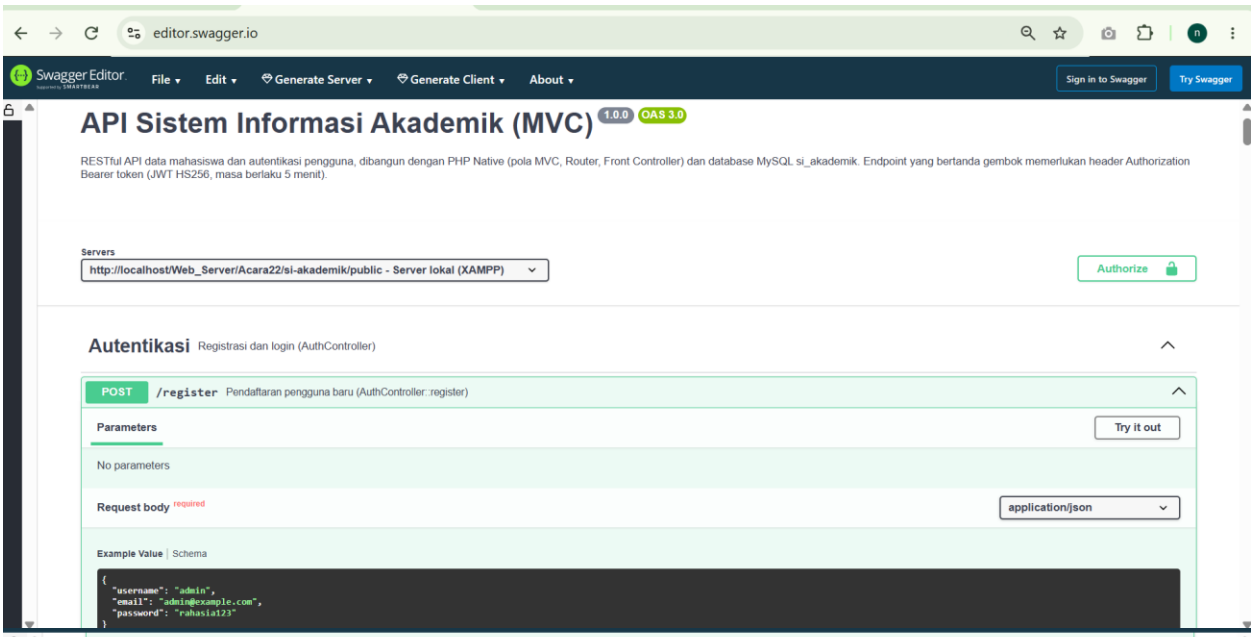


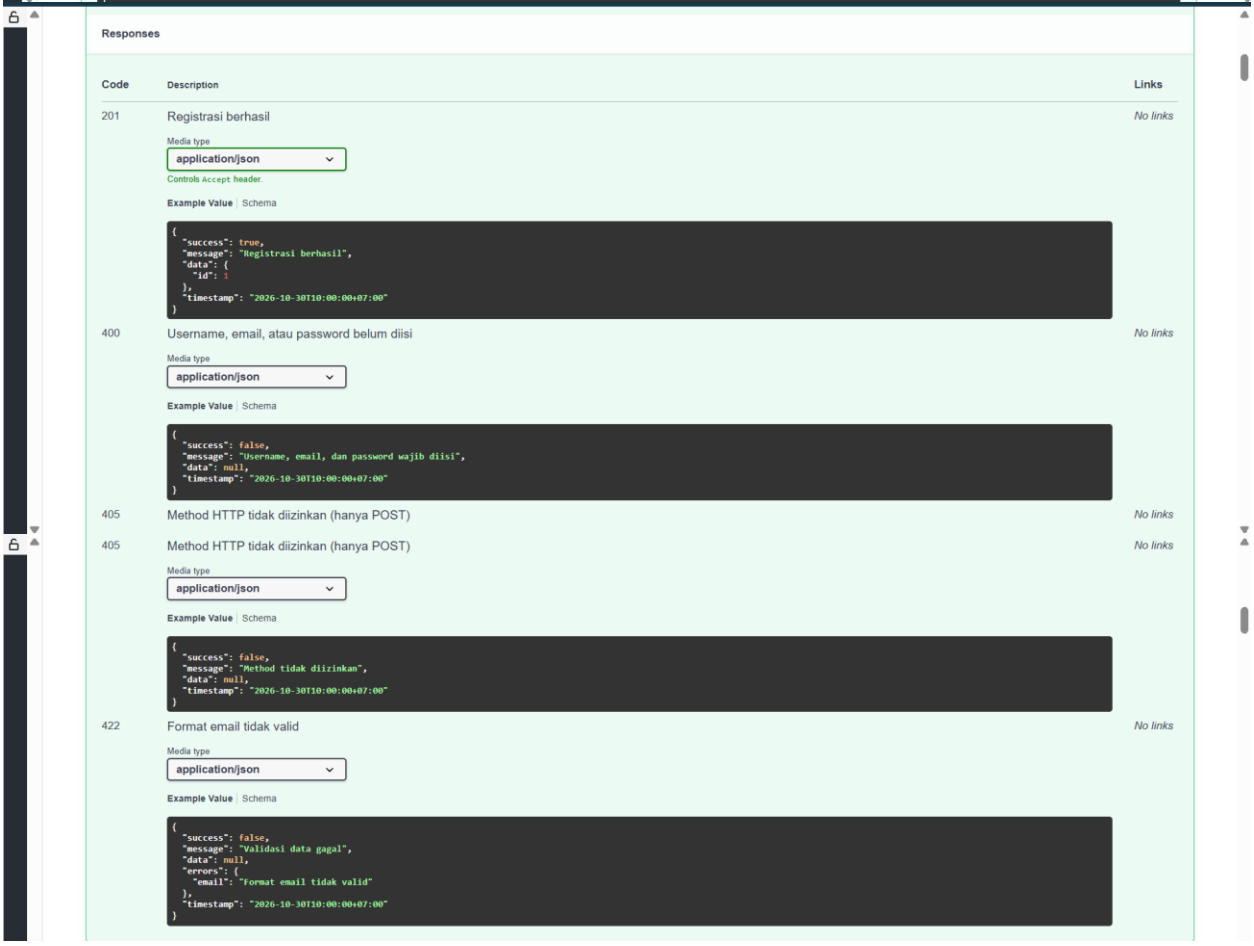
Tabel berikut merangkum seluruh method pada path /mahasiswa.

Method	Controller	Autentikasi	Parameter	Response
GET	index	Publik	?id (opsional)	200, 404
POST	store	Bearer token	Request body	201, 400, 401, 409, 422
PUT	update	Bearer token	?id wajib dan request body	200, 400, 401, 404, 409, 422
DELETE	destroy	Bearer token	?id wajib	200, 400, 401, 404

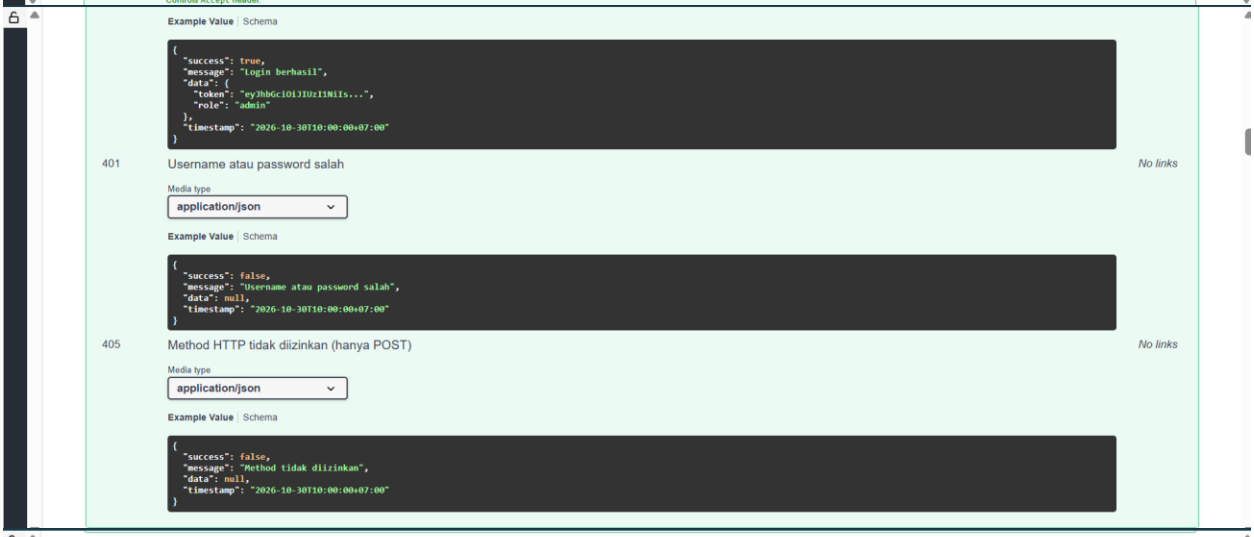
Langkah 4 - Memvisualisasikan Dokumentasi

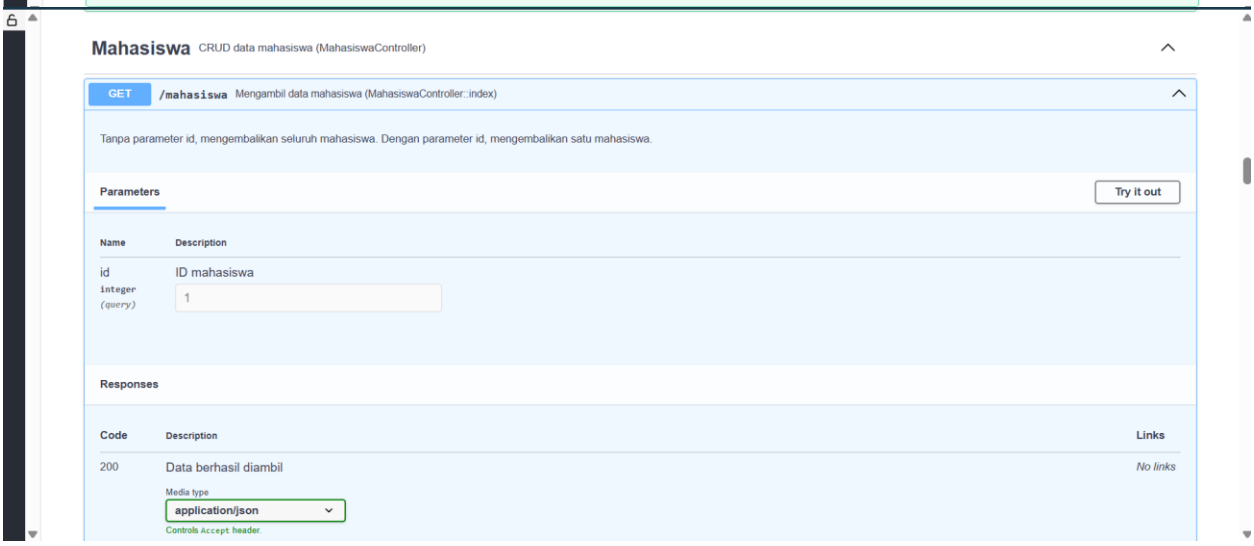
1. Membuka https://editor.swagger.io pada browser.
2. Menempelkan seluruh isi openapi.yaml pada panel kiri editor, lalu mengamati pratinjau pada panel kanan.
3. Menyimpan tangkapan layar hasil visualisasi sebagai lampiran dokumentasi proyek pada folder docs/.


editor.swagger.io
Swagger Editor
File
Edit
Generate Server
Generate Client
About
Sign in to Swagger
Try Swagger
API Sistem Informasi Akademik (MVC)
1.0.0
OAS 3.0
RESTful API data mahasiswa dan autentikasi pengguna, dibangun dengan PHP Native (pola MVC, Router, Front Controller) dan database MySQL si_akademik. Endpoint yang bertanda gembok memerlukan header Authorization Bearer token (JWT HS256, masa berlaku 5 menit).
Servers
http://localhost/Web_Server/Acara22/si-akademik/public - Server lokal (XAMPP)
Authorize
Autentikasi
Registrasi dan login (AuthController)
POST
/register
Pendaftaran pengguna baru (AuthController::register)
Parameters
Try it out
No parameters
Request body
required
application/json
Example Value
Schema
{
"username": "admin",
"email": "admin@example.com",
"password": "rahasia123"
}


Responses
Code
Description
Links
201
Registrasi berhasil
No links
Media type
application/json
Controls Accept header.
Example Value
Schema
{
"success": true,
"message": "Registrasi berhasil",
"data": {
"id": 1
},
"timestamp": "2026-10-30T10:00:00+07:00"
}
400
Username, email, atau password belum diisi
No links
Media type
application/json
Example Value
Schema
{
"success": false,
"message": "Username, email, dan password wajib diisi",
"data": null,
"timestamp": "2026-10-30T10:00:00+07:00"
}
405
Method HTTP tidak diizinkan (hanya POST)
No links
405
Method HTTP tidak diizinkan (hanya POST)
No links
Media type
application/json
Example Value
Schema
{
"success": false,
"message": "Method tidak diizinkan",
"data": null,
"timestamp": "2026-10-30T10:00:00+07:00"
}
422
Format email tidak valid
No links
Media type
application/json
Example Value
Schema
{
"success": false,
"message": "Validasi data gagal",
"data": null,
"errors": {
"email": "Format email tidak valid"
},
"timestamp": "2026-10-30T10:00:00+07:00"
}


POST /login Autentikasi pengguna (AuthController::login)
Menghasilkan token JWT (HS256) yang berlaku selama 5 menit.
Parameters
Try it out
No parameters
Request body required
application/json
Example Value | Schema
{
"username": "admin",
"password": "rahasia123"
}
Responses
Code
Description
Links
200
Login berhasil
No links
Media type
application/json
Controls Accept header.


Example Value | Schema
{
"success": true,
"message": "Login berhasil",
"data": {
"token": "eyJhbGciOiJIUzI1NiIs...",
"role": "admin"
},
"timestamp": "2026-10-30T10:00:00+07:00"
}
401
Username atau password salah
No links
Media type
application/json
Example Value | Schema
{
"success": false,
"message": "Username atau password salah",
"data": null,
"timestamp": "2026-10-30T10:00:00+07:00"
}
405
Method HTTP tidak diizinkan (hanya POST)
No links
Media type
application/json
Example Value | Schema
{
"success": false,
"message": "Method tidak diizinkan",
"data": null,
"timestamp": "2026-10-30T10:00:00+07:00"
}


Mahasiswa CRUD data mahasiswa (MahasiswaController)
GET /mahasiswa Mengambil data mahasiswa (MahasiswaController::index)
Tanpa parameter id, mengembalikan seluruh mahasiswa. Dengan parameter id, mengembalikan satu mahasiswa.
Parameters
Try it out
Name
Description
id
integer
(query)
ID mahasiswa
1
Responses
Code
Description
Links
200
Data berhasil diambil
No links
Media type
application/json
Controls Accept header.

Example Value | Schema


```
{
  "success": true,
  "message": "string",
  "timestamp": "2026-11-01T11:10:49.837Z",
  "data": [
    {
      "id": 1,
      "nim": "230001",
      "nama": "Nila Agustin",
      "email": "nila@example.com",
      "prodi_id": 1,
      "prodi": "Teknik Informatika",
      "angkatan": 2023,
      "status": "aktif"
    }
  ]
}
```


404 Data tidak ditemukan — No links

Media type: application/json

Example Value | Schema


```
{
  "success": false,
  "message": "Data tidak ditemukan",
  "data": null,
  "timestamp": "2026-10-30T10:00:00+07:00"
}
```


POST /mahasiswa Menambahkan data mahasiswa (MahasiswaController::store)

Parameters — Try it out

No parameters

Request body required — application/json

Example Value | Schema


```
{
  "nim": "230001",
  "nama": "Nila Agustin",
  "email": "nila@example.com",
  "prodi_id": 1,
  "angkatan": 2023,
  "status": "aktif"
}
```


Responses

Code	Description	Links
201	Data berhasil ditambahkan	No links

Media type: application/json

Controls Accept header.

Example Value | Schema


```
{
  "success": true,
  "message": "Data berhasil ditambahkan",
  "data": {
    "id": 1
  },
  "timestamp": "2026-10-30T10:00:00+07:00"
}
```


400 Field wajib belum lengkap — No links

Media type: application/json

Example Value | Schema


```
{
  "success": false,
  "message": "Field wajib belum lengkap",
  "data": null,
  "errors": {
    "nama": "Field nama wajib diisi"
  },
  "timestamp": "2026-10-30T10:00:00+07:00"
}
```


401 Token tidak ditemukan, tidak valid, atau kedaluwarsa — No links

Media type: application/json — Examples: Token tidak dikirim

Example Value | Schema


```
{
  "success": false,
  "message": "Token tidak ditemukan",
  "data": null,
  "timestamp": "2026-10-30T10:00:00+07:00"
}
```

409 NIM sudah terdaftar — No links

Media type: application/json

Example Value | Schema

```
{
  "success": false,
  "message": "NIM sudah terdaftar",
  "data": null,
  "errors": {
    "nim": "NIM sudah terdaftar"
  },
  "timestamp": "2026-10-30T10:00:00+07:00"
}
```

422 Data lengkap tetapi gagal validasi. Kemungkinan isi errors: nim (bukan 6 digit angka), email (format salah), status (selain aktif/cuti/lulus), prodi_id (tidak ditemukan). — No links

Media type: application/json

Example Value | Schema

```
{
  "success": false,
  "message": "Validasi data gagal",
  "data": null,
  "errors": {
    "nim": "Format NIM tidak valid, harus 6 digit angka"
  },
  "timestamp": "2026-10-30T10:00:00+07:00"
}
```

PUT /mahasiswa Mengubah data mahasiswa (MahasiswaController::update)

Seluruh field wajib dikirim ulang, sama seperti POST.

Parameters — Try it out

Name	Description
id * required integer (query)	ID mahasiswa (angka) 1

Request body required — application/json

Example Value | Schema

```
{
  "nim": "230001",
  "nama": "Nila Agustin Prima",
  "email": "nila@example.com",
  "prodi_id": 1,
  "angkatan": 2023,
  "status": "aktif"
}
```

Responses

Code	Description	Links
200	Data berhasil diperbarui Media type: application/json Controls Accept header. Example Value | Schema	No links

```
{
  "success": true,
  "message": "Data berhasil diperbarui",
  "data": null,
  "timestamp": "2026-10-30T10:00:00+07:00"
}
```

400 ID kosong/bukan angka, atau field wajib belum lengkap — No links

Media type: application/json

Example Value | Schema

```
{
  "success": false,
  "message": "ID wajib diisi",
  "data": null,
  "timestamp": "2026-10-30T10:00:00+07:00"
}
```

401 Token tidak ditemukan, tidak valid, atau kedaluwarsa — No links

Media type: application/json — Examples: Token tidak dikirim

Example Value | Schema

```
{
  "success": false,
  "message": "Token tidak ditemukan",
  "data": null,
  "timestamp": "2026-10-30T10:00:00+07:00"
}
```

404 Data tidak ditemukan — No links

Media type: application/json

Example Value | Schema

```
{
  "success": false,
  "message": "Data tidak ditemukan",
  "data": null,
  "timestamp": "2026-10-30T10:00:00+07:00"
}
```

409 NIM dipakai mahasiswa lain — No links

Media type: application/json

Example Value | Schema

Swagger Editor — File ▾ Edit ▾ Generate Server ▾ Generate Client ▾ About ▾ — Sign in to Swagger — Try Swagger

```
{
  "success": false,
  "message": "NIM sudah dipakai mahasiswa lain",
  "data": null,
  "errors": {
    "nim": "NIM sudah dipakai mahasiswa lain"
  },
  "timestamp": "2026-10-30T10:00:00+07:00"
}
```

422 Data lengkap tetapi gagal validasi. Kemungkinan isi errors: nim (bukan 6 digit angka), email (format salah), status (selain aktif/cuti/lulus), prodi_id (tidak ditemukan). — No links

Media type: application/json

Example Value | Schema

```
{
  "success": false,
  "message": "Validasi data gagal",
  "data": null,
  "errors": {
    "nim": "Format NIM tidak valid, harus 6 digit angka"
  },
  "timestamp": "2026-10-30T10:00:00+07:00"
}
```

DELETE /mahasiswa Menghapus data mahasiswa (MahasiswaController::destroy)

Parameters — Try it out

Name	Description
id * required integer (query)	ID mahasiswa (angka) 1

Responses

Code	Description	Links
200	Data berhasil dihapus Media type: application/json Controls Accept header. Example Value | Schema	No links

```
{
  "success": true,
  "message": "Data berhasil dihapus",
  "data": null,
  "timestamp": "2026-10-30T10:00:00+07:00"
}
```

400 ID kosong atau bukan angka — No links

Media type: application/json

Example Value | Schema

```
{
  "success": false,
  "message": "ID wajib diisi",
  "data": null,
  "timestamp": "2026-10-30T10:00:00+07:00"
}
```

401 Token tidak ditemukan, tidak valid, atau kedaluwarsa — No links

Media type: application/json — Examples: Token tidak dikirim

Example Value | Schema

```
{
  "success": false,
  "message": "Token tidak ditemukan",
  "data": null,
  "timestamp": "2026-10-30T10:00:00+07:00"
}
```

404 Data tidak ditemukan — *No links*

Media type: application/json

Example Value | Schema

```
{
  "success": false,
  "message": "Data tidak ditemukan",
  "data": null,
  "timestamp": "2026-10-30T10:00:00+07:00"
}
```

Schemas

RegisterRequest {
- username* > [...]
- email* > [...]
- password* > [...]

}

LoginRequest {
- username* > [...]
- password* > [...]

}

MahasiswaInput {
- nim* > [...]
- nama* > [...]
- email* > [...]
- prodi_id* > [...]
- angkatan* > [...]
- status > [...]

}

Mahasiswa {
- id > [...]
- nim > [...]
- nama > [...]
- email > [...]
- prodi_id > [...]
- prodi > [...]
- angkatan > [...]
- status > [...]

}

BaseResponse {
- success > [...]
- message > [...]
- timestamp > [...]

}

SuccessResponse {
- success > [...]
- message > [...]
- timestamp > [...]
- data > {...}
  nullable: true
  example:

}

ErrorResponse {
- success > [...]
- message > [...]
- timestamp > [...]
- data > {...}
  nullable: true
  example:
- errors > {...}

}



3.2 Tugas Praktik Mandiri

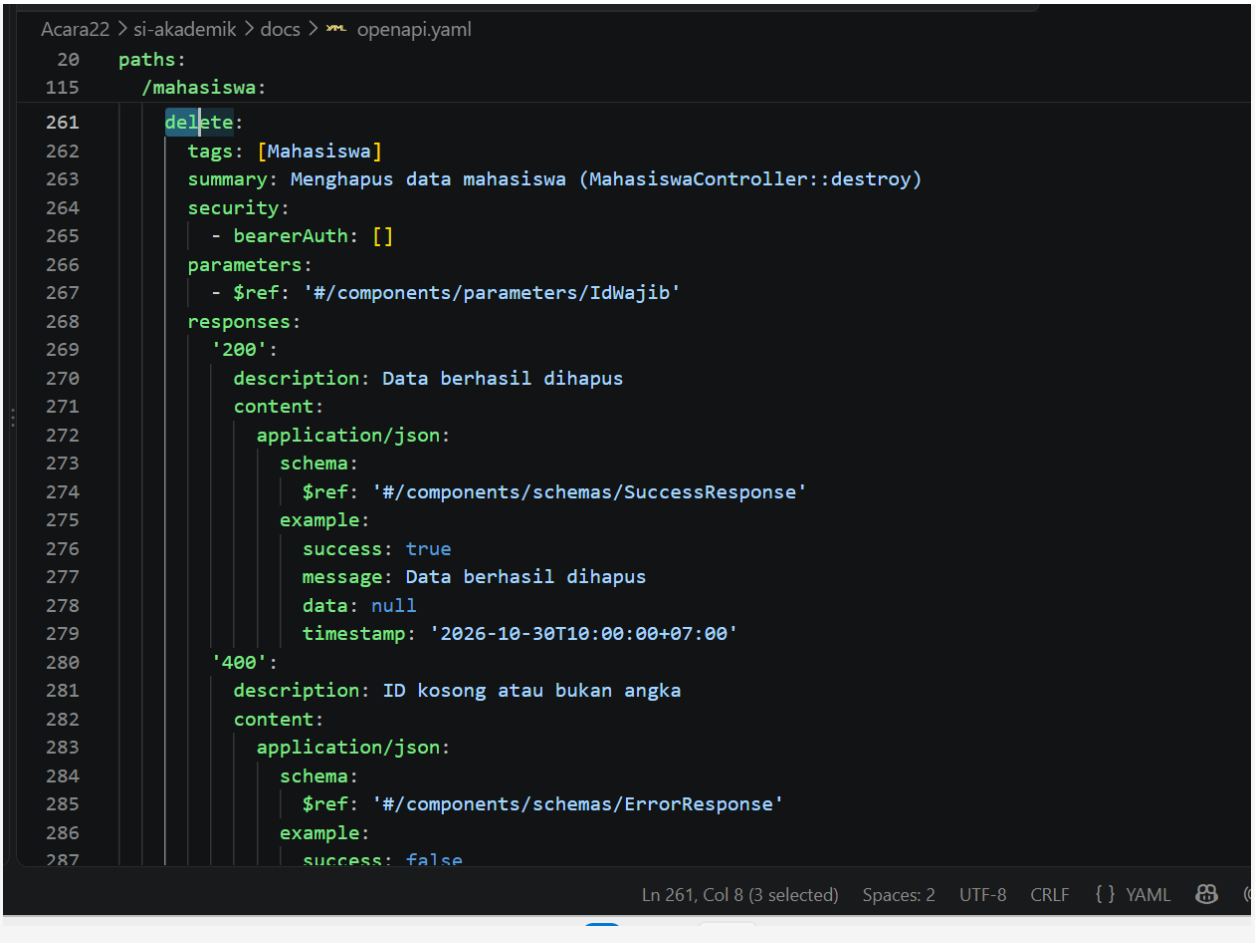
Tugas 1 - Melengkapi seluruh path pada openapi.yaml

Path yang belum ada pada contoh modul dilengkapi mengikuti pola yang sama, yaitu method PUT dan DELETE pada /mahasiswa serta path /register.

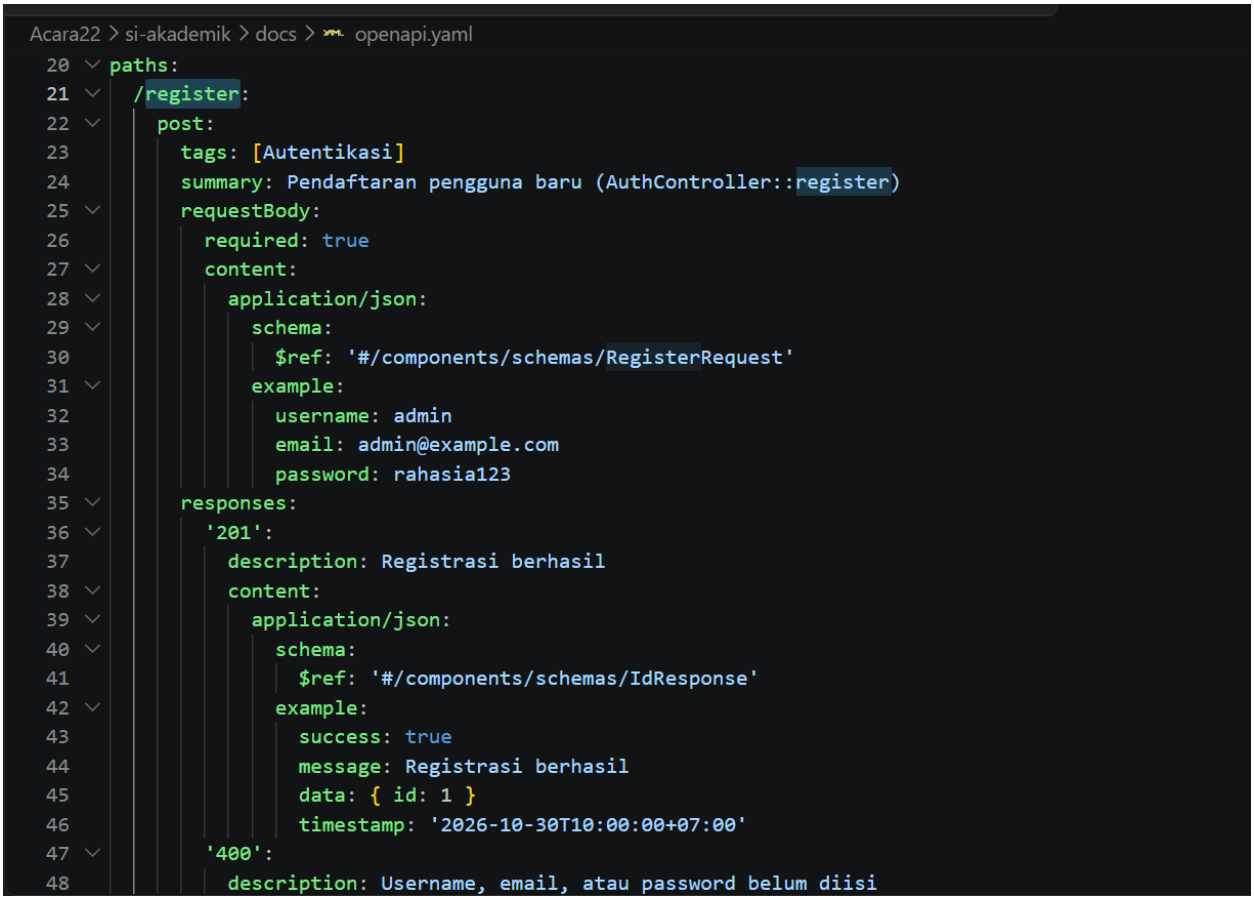
PUT dan DELETE memerlukan parameter query id yang wajib, dideklarasikan sekali pada components/parameters lalu dipakai ulang.

Acara22 > si-akademik > docs > openapi.yaml

```yaml
paths:
  /mahasiswa:
    put:
      tags: [Mahasiswa]
      summary: Mengubah data mahasiswa (MahasiswaController::update)
      description: Seluruh field wajib dikirim ulang, sama seperti POST.
      security:
        - bearerAuth: []
      parameters:
        - $ref: '#/components/parameters/IdWajib'
      requestBody:
        required: true
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/MahasiswaInput'
            example:
              nim: '230001'
              nama: Nila Agustin Prima
              email: nila@example.com
              prodi_id: 1
              angkatan: 2023
              status: aktif
      responses:
        '200':
          description: Data berhasil diperbarui
          content:
            application/json:
              schema:
```

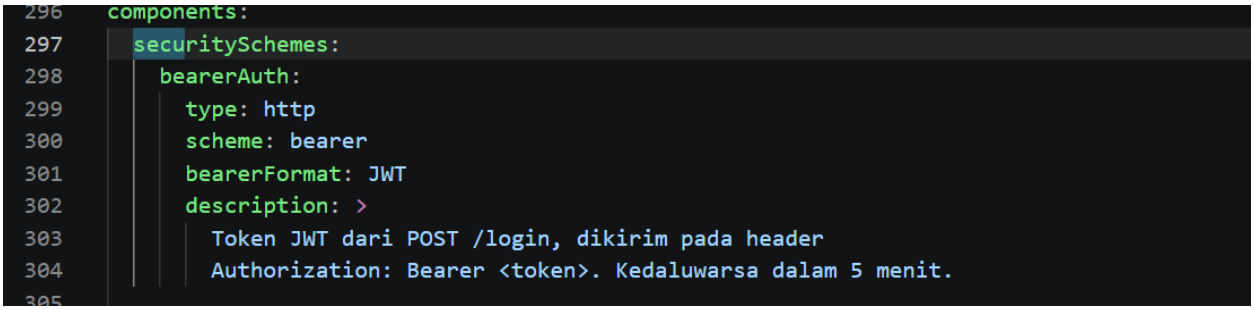


Path /register memiliki method post dengan field wajib username, email, dan password. Response yang didokumentasikan adalah 201 (berisi id pengguna baru), 400 (field kosong), 405, dan 422 (format email tidak valid).



Tugas 2 - Menambahkan securitySchemes bertipe http/bearer

Pada bagian components ditambahkan skema keamanan bearerAuth bertipe http dengan scheme: bearer dan bearerFormat: JWT. Skema ini diterapkan pada POST, PUT, dan DELETE /mahasiswa melalui security: [{ bearerAuth: [] }]. Pada Swagger Editor, tombol Authorize muncul sehingga token hasil POST /login dapat dimasukkan sekali untuk seluruh endpoint terproteksi.



BAB 4 KESIMPULAN

Berdasarkan praktikum yang telah dilakukan, dokumentasi API si-akademik berhasil dikonversi dari format Markdown pada Acara 21 menjadi berkas docs/openapi.yaml yang mengikuti standar OpenAPI Specification 3.0. OpenAPI merupakan format standar industri untuk mendeskripsikan REST API dalam YAML atau JSON yang dapat dibaca manusia maupun mesin, sehingga dokumentasi memiliki struktur baku dan dapat divalidasi secara otomatis oleh editor. Path yang didokumentasikan mengikuti resource pada peta Router, yaitu /register, /login, dan /mahasiswa dengan method GET, POST, PUT, dan DELETE, lengkap dengan parameter, request body, serta response beserta kode statusnya.

Penambahan securitySchemes bertipe http/bearer pada bagian components membuat endpoint yang terproteksi token JWT dapat dibedakan dengan jelas dari endpoint publik. Pada tampilan Swagger Editor, endpoint POST, PUT, dan DELETE /mahasiswa ditandai ikon gembok, dan tombol Authorize memungkinkan token hasil POST /login dimasukkan sekali untuk seluruh endpoint tersebut. Pemakaian components dan $ref untuk schema, response, dan parameter juga membuat dokumen lebih ringkas dan konsisten karena struktur yang sama tidak perlu ditulis ulang pada setiap endpoint.

Dibandingkan dokumentasi Markdown pada Acara 21 yang berupa teks statis, dokumentasi OpenAPI yang divisualisasikan melalui Swagger Editor dan Swagger UI bersifat interaktif karena setiap endpoint dapat langsung dicoba (try-it-out) tanpa berpindah ke Postman. Dengan demikian, OpenAPI lebih sesuai digunakan sebagai rujukan resmi bagi pengembang frontend maupun pihak ketiga yang akan mengintegrasikan API si-akademik.

LAMPIRAN